QRShield

QR Code Threat Detection Platform

QRShield is a high-performance, premium web application designed to analyze QR codes and URLs for potential security threats in real-time. By leveraging a custom backend rule engine, it evaluates payloads for malicious patterns, suspicious keywords, and structural anomalies, presenting the results in a clean, enterprise-grade dashboard.

Key Features

- **Live Camera Scanner:** Real-time QR code scanning directly from the browser using the HTML5 Camera API.
- **Custom Rule Engine:** Evaluates URLs against multiple security heuristics (e.g., IP address masking, excessive subdomains, suspicious keywords, URL shorteners).
- **Risk Scoring System:** Generates a granular threat score (0-100) and categorizes the payload as SAFE, SUSPICIOUS, or MALICIOUS.
- **Structured Security Reports:** Provides detailed, lecturer-friendly breakdowns of exactly which security checks passed and which failed.
- **Premium UI:** A bright, high-end fintech-style interface with responsive layouts, smooth animations, and clean typography.

Tech Stack

Backend

- **FastAPI:** High-performance web framework for building the API and serving the application.
- **Python 3.11+:** Core backend programming language.
- **Uvicorn:** ASGI web server for fast, asynchronous execution.
- **Pydantic:** Data validation and settings management.
- **Poetry:** Dependency management and virtual environment isolation.

Frontend

- **HTML5 / Vanilla JavaScript / CSS3:** Lightweight, framework-free frontend for maximum speed and minimal overhead.
- **Html5-Qrcode:** Open-source library for client-side camera access and QR decoding.
- **Font Awesome v6:** Vector icons for the premium dashboard UI.

Architecture & Component Flow

QRShield follows a lightweight Client-Server architecture designed for speed and modularity:

1. **Client-Side Capture (app.js & index.html):** The user inputs a URL manually or scans a physical QR code using their device camera. The decoded payload is intercepted by the JavaScript frontend.
2. **Asynchronous API Call:** The frontend sends the raw string data to the backend via a POST request to the /analyze endpoint.
3. **Payload Detection (payload_detector.py):** The backend intercepts the payload and categorizes it (e.g., standard URL, UPI payment link, or plain TEXT).
4. **Rule Engine Processing (rule_engine.py):** If a URL is detected, it is passed through the security engine. The engine applies multiple regex and structural checks:
    - *Protocol Check* (HTTP vs HTTPS)
    - *Domain Obfuscation* (Presence of @, IP addresses, or URL shorteners)
    - *Structural Anomalies* (Length limits, excessive hyphens/digits)
    - *Phishing Indicators* (Suspicious keywords like "login", "verify", "secure")
5. **Response & Rendering:** The backend aggregates the rule triggers, calculates a cumulative threat score, and returns a structured JSON report. The frontend dynamically parses this JSON and visually renders the passing/failing metrics into the premium dashboard.

Installation & Running

QRShield uses Poetry to make dependency management effortless. You only need two commands to get the entire platform running locally.

1. Install Dependencies This command reads the pyproject.toml and installs the exact versions of the required packages in an isolated environment.

Bash

Plain `poetry install`

2. Start the Server This command spins up the Uvicorn server with hot-reloading enabled.

Bash

Plain `poetry run uvicorn app.server:app --reload`

Access the App: Open your browser and navigate to: http://127.0.0.1:8000